



UNIVERSIDAD
POLITÉCNICA
DE MADRID



Planificador de Viajes

***Proyecto de la asignatura Taller de Programación
Convocatoria ordinaria, curso 2025-2026***

E. T. S. de Ingeniería de Sistemas Informáticos
Universidad Politécnica de Madrid

10 de noviembre de 2025

Licencias

© [2025] [Jorge Dueñas Lerín] [Jordi Burguet Castell]

Este material se distribuye bajo una licencia  Atribución/Reconocimiento-NoComercial-CompartirIgual 4.0 Internacional de Creative Commons. Para ver una copia de esta licencia, visite <https://creativecommons.org/licenses/by-nc-sa/4.0/deed.es>.

Usted es libre de:

- Compartir — copiar y redistribuir el material en cualquier medio o formato
- Adaptar — remezclar, transformar y construir a partir del material

Bajo los siguientes términos:

-  Atribución — Usted debe dar crédito de manera adecuada, brindar un enlace a la licencia e indicar si se han realizado cambios. Puede hacerlo de cualquier manera razonable, pero no de una manera que sugiera que tiene el apoyo del licenciante o lo recibe por el uso que hace.
-  NoComercial — Usted no puede utilizar el material con fines comerciales.
-  CompartirIgual — Si remezcla, transforma o crea a partir del material, debe distribuir su contribución bajo la misma licencia que el original.
- No hay restricciones adicionales — No puede aplicar términos legales ni medidas tecnológicas que restrinjan legalmente a otras a hacer cualquier uso permitido por la licencia.

Atribución

Este trabajo se basa en una versión anterior elaborada por Raúl Lara Cabrera (2024), también distribuida bajo la misma licencia Creative Commons.

El enunciado de esta práctica se ha elaborado con $\text{\LaTeX}$, basándose en la plantilla “Sullivan Business Report” de Vel, publicada en <https://www.LaTeXTemplates.com>, que se distribuye bajo una licencia  Atribución/Reconocimiento-NoComercial-CompartirIgual 4.0 Internacional de Creative Commons.

Contacto

Para cualquier duda o comentario sobre esta práctica, por favor, contacta con los profesores de la asignatura.

Control de versiones

v1.0	09-12-2025	Primera versión.
v0.3	24-10-2025	Modificada IO.
v0.2	22-10-2025	Boceto completo.
v0.1	17-10-2025	Primer boceto de la práctica.

Índice

1 Introducción	4
1.1 Preparación del entorno de desarrollo	4
2 Descripción y requisitos del proyecto	6
2.1 Descripción del proyecto	6
2.2 Componentes a desarrollar	6
2.3 La función principal main	28
2.4 Casos de prueba	30
3 Criterios de evaluación	31
3.1 Casos de prueba	31
3.2 Diseño y estructura del código	31
3.3 Documentación y comentarios	32
3.4 Errores graves	32
Referencias	33

1 Introducción

“Viajar es descubrir que todos están equivocados sobre otros países...”
— Aldous Huxley

Organizar y planificar un viaje puede ser tan emocionante como abrumador: elegir destinos, actividades y horarios puede convertirse en un rompecabezas. Hemos decidido crear una aplicación Java para ayudar a los usuarios a organizar viajes. En esta práctica desarrollarás una aplicación de consola que permitirá gestionar un catálogo de actividades y posteriormente organizarlas en itinerarios de viaje.

Durante el desarrollo del proyecto pondrás en práctica conceptos esenciales de programación, como lectura y escritura por consola, manipulación de arrays, diseño de funciones y estructuras de control. Al finalizar, habrás creado una herramienta funcional capaz de:

1. Almacenar y consultar actividades turísticas con detalles como nombre, descripción, precio y duración.
2. Crear itinerarios de viaje personalizados, asegurando que las actividades no se solapen en horarios.
3. Guardar y cargar datos desde archivos de texto, facilitando la persistencia de la información.

Además de ayudarte a dominar Java, este proyecto te impulsará a pensar con claridad y a resolver problemas reales de forma práctica y eficiente mediante la programación.

1.1 Preparación del entorno de desarrollo

Para el desarrollo de este proyecto, podrás utilizar cualquier entorno de desarrollo integrado (IDE) que prefieras, aunque el soporte completo será ofrecido únicamente para IntelliJ IDEA, un potente IDE de JetBrains ampliamente utilizado en la industria y de código libre (licencia Apache 2.0) en su versión Community Edition, y para el cual, además, tenemos licencia educativa¹ toda la comunidad universitaria.

El esqueleto del proyecto se encuentra disponible en un repositorio del GitLab de la escuela. Todos los estudiantes tienen una cuenta creada en la plataforma y pueden acceder con su número de matrícula y su contraseña <https://gitlab.etsisi.upm.es/>.

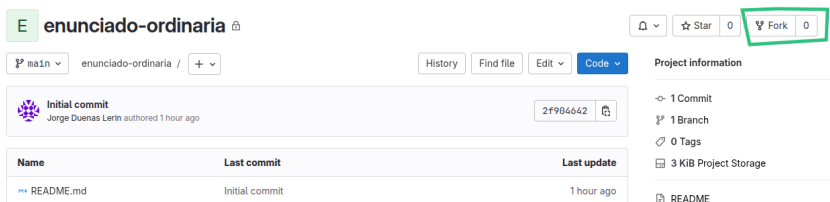
¹Licencias educativas gratuitas. Asistencia a la comunidad [2]

El repositorio de la práctica pertenece a los profesores de la asignatura con lo que no podrás modificarlo. Este repositorio está disponible de forma pública y es software libre con lo que puedes crear tu propia copia y hacer modificaciones sobre ella. Esta acción recibe el nombre de fork (bifurcación). Para ello, sigue los siguientes pasos:

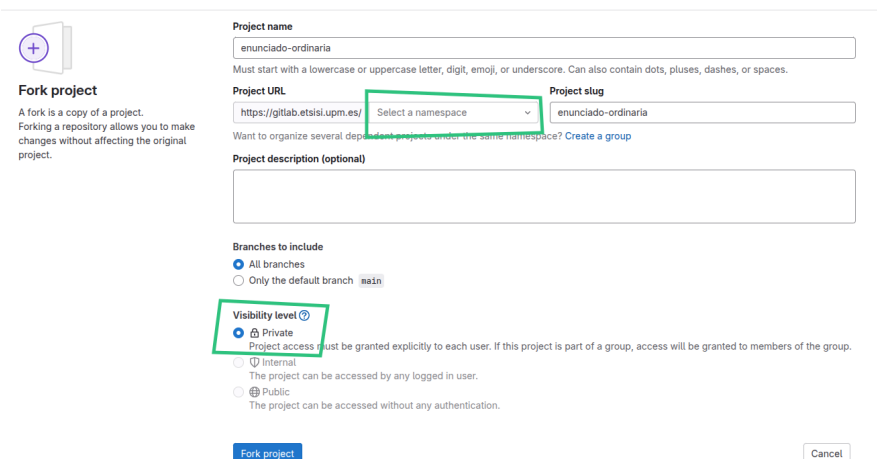
1º Accede al repositorio del proyecto:

<https://gitlab.etsisi.upm.es/tallerdeprogramacion/2526/enunciado-ordinaria>

2º Haz un fork del proyecto:



3º Configura la visibilidad y asócialo a tu usuario (Solo lo hará 1 persona del grupo)



4º Clona tu nuevo repositorio en tu equipo y comienza a trabajar en tu copia del proyecto.

- Si utilizas IntelliJ IDEA, puedes clonar el repositorio directamente desde el IDE. Para ello, selecciona la opción “Clone Repository” en el menú de inicio y pega la URL de tu repositorio.
- Si vas a trabajar desde la línea de comandos, puedes clonar el repositorio con el comando `git clone` seguido de la URL de tu repositorio.

5º No te olvides de *añadir al resto de integrantes como miembros* en tu repositorio (en Manage – Members).

Tras estas acciones podrás comenzar a utilizar el control de

versiones git y sincronizar el trabajo sobre el código.

IMPORTANTE: *queda fuera del alcance de esta asignatura el uso de control de versiones. Existen prácticas opcionales en el Moodle. El uso de git es opcional aunque lo **recomendamos**.*

Para finalizar esta sección, un último detalle. El proyecto se ha creado con Maven², un sistema de automatización de compilación y gestión de dependencias. Si utilizas IntelliJ IDEA, el IDE se encargará de importar el proyecto y configurar Maven automáticamente. Si trabajas desde la línea de comandos, puedes compilar el proyecto con el comando `mvn compile`. En el caso de que quieras gestionar las dependencias del proyecto manualmente, en la Tabla 1 se detallan las dependencias necesarias para el proyecto

²Redmond [3]

Tabla 1: Dependencias del proyecto.

Dep.	Versión
JDK	22
JUnit	5.8.1

2 Descripción y requisitos del proyecto

El objetivo de este proyecto es desarrollar un sistema en Java que permita gestionar y planificar viajes. Los estudiantes deberán implementar varias clases y métodos para cumplir con los requisitos especificados.

2.1 Descripción del proyecto

La herramienta a desarrollar es lo que se conoce como una aplicación de línea de comandos (CLI, por sus siglas en inglés). La interfaz de usuario será textual, sin gráficos ni interacción con el ratón. El programa se ejecutará en la terminal de un sistema operativo, lo que se conoce como Text-based User Interface (TUI)³. Para la realización del proyecto se proporciona un esquema de clases y métodos que los estudiantes deberán completar. Se usarán los conceptos de programación ya trabajados en la asignatura de Fundamentos de Programación.

³Aunque pueda parecer algo del pasado, las TUI están a la orden del día. Echa un vistazo al siguiente [enlace](#)

2.2 Componentes a desarrollar

La arquitectura del sistema se divide en cinco componentes, como se puede observar en la Tabla 2. Cada componente tie-

ne una funcionalidad específica y se encarga de gestionar una parte del sistema.

Tabla 2: Componentes a desarrollar.

#	Componente	Descripción
1	Actividad	Representa una actividad a realizar dentro de un viaje. Una actividad almacena el nombre, la descripción, el precio, la duración, etc. Se puede ver el listado en el apartado correspondiente. Proporciona además métodos para obtener información sobre la actividad.
2	CatalogoActividades	Almacena un conjunto de actividades. Permite agregar, buscar y eliminar actividades. Permite guardar y cargar actividades desde archivos.
4	Viaje	Permite planificar viajes de distintos días de duración sin que las actividades se solapen. También permite guardar y cargar el plan del viaje a través de un archivo.
5	InterfazUsuario	Gestiona la interacción con el usuario a través de un menú en consola.
6	Utilidades	Proporciona métodos de utilidad para la entrada de datos por teclado.

A continuación se detallan los requisitos de cada componente.

2.2.1 La actividad

El componente Actividad representa una actividad a realizar dentro de un viaje. Cada actividad tiene un nombre, una descripción, un precio y una duración (en minutos). Además, gestiona dos *arrays* (a veces en castellano se les dice “arreglos”): un array de *recursos* asociados (por ejemplo, enlaces, materiales, reservas) y un array de *comentarios* (observaciones o notas de usuario). El componente debe proporcionar métodos para agregar recursos y comentarios a la actividad, así como para obtener información de la misma.

El número máximo de recursos y comentarios que puede tener una actividad se especifican como parámetros de ejecución. La clase Actividad define constantes para representar los posibles resultados de las operaciones de agregado⁴:

Listado 1: Constantes de códigos de error en la clase Actividad.

El componente Actividad es fundamental para el funcionamiento del sistema, pues se usa tanto en el `CatalogoActividades` como en `Viaje`. Asegúrate de implementar correctamente los métodos necesarios.

⁴ Este enfoque permite separar la lógica de negocio (en `Actividad`) de la presentación (en `InterfazUsuario`), facilitando el mantenimiento y las pruebas del código.

```
1 public static final int EXITO = 0;
2 public static final int ERROR_VALOR_INVALIDO = 1;
3 public static final int ERROR_RECURSOS_COMPLETOS = 2;
4 public static final int ERROR_COMENTARIOS_COMPLETOS = 3;
```

La funcionalidad que nos permite agregar un recurso se encapsula en el método que recibe como parámetro el recurso a añadir.

```
public int agregarRecurso(String recurso)
```

La funcionalidad de agregar un comentario se encapsula en el método que recibe como parámetro el comentario.

```
public int agregarComentario(String comentario)
```

Ambos métodos devuelven un código entero que indica el resultado de la operación⁵. Es responsabilidad de la clase InterfazUsuario interpretar estos códigos y mostrar los mensajes apropiados al usuario. Por ejemplo, cuando se intenta agregar más recursos o comentarios de los permitidos, InterfazUsuario mostrará:

Listado 2: Mensajes de error mostrados por InterfazUsuario al interpretar los códigos de error.

- 1 No se pueden añadir más recursos.
 - 2 No se pueden añadir más comentarios.
-

Una funcionalidad adicional que se pide es la de obtener la información completa de la actividad. Esta funcionalidad se encapsula en el método:

```
public String toString()
```

Este método devuelve una cadena de caracteres con la información de la actividad en el siguiente formato⁶:

Listado 3: Formato de representación textual de una actividad.

- 1 Actividad: nombre de la actividad
- 2 Descripción: descripción de la actividad
- 3 Precio: 123.45 €

⁵EXITO si se ha realizado la operación correctamente, ERROR_VALOR_INVALIDO si el parámetro es null o vacío, ERROR_RECURSOS_COMPLETOS o ERROR_COMENTARIOS_COMPLETOS si se ha superado el máximo permitido

⁶Ten en cuenta los espacios y saltos de línea en la salida, si no lo hace los test sobre la clase fallarán.

El número de recursos y comentarios puede variar. En este ejemplo hay 2 recursos y 3 comentarios

Recomendamos utilizar **StringBuilder** en lugar de la concatenación de cadenas, especialmente cuando manipulas grandes cantidades de texto


```
4 Duración: 1h 30min
5 Recursos:
6 - Recurso 1
7 - Recurso 2
8 Comentarios:
9 1. Comentario 1
10 2. Comentario 2
11 3. Comentario 3
```

Para guardar las actividades a partir de un fichero de texto se usa una representación textual **simplificada** de las actividades. Esta representación se genera en el método

```
public String toRawString()
```

y devuelve la información de la actividad, incluyendo el nombre, la descripción, el precio, la duración, los recursos y los comentarios, en el siguiente formato compacto⁷:

Listado 4: Formato de representación textual simplificada y compacta de una actividad

```
1 Nombre de la actividad
2 Descripción de la actividad
3 123.45
4 90
5 Recurso 1
6 Recurso n
7 COMENTARIOS
8 Comentario 1
9 Comentario m
10 -----
```

⁷ Presta atención a las diferencias con la representación textual completa mostrada en el Listado 3. Fíjate también en el separador de actividades al final del texto.

Para cargar desde un fichero y generar una actividad tendremos un método estático que espera recibir un `BufferedReader` ya abierto desde el que se están cargando actividades (ver `CatalogoActividades`).

```
public static Actividad fromBufferedReader(BufferedReader reader, int maxRecursos, int maxComentarios)
```

Este método lee la información de una actividad desde el `BufferedReader` proporcionado y devuelve un objeto `Actividad` con la información leída. Los parámetros `maxRecursos` y `maxComentarios` indican el número máximo de recursos y comentarios que

puede contener la actividad. Si el archivo contiene más recursos o comentarios de los especificados, la información se truncará.

Además de los métodos anteriores, el componente Actividad debe proporcionar los métodos auxiliares que son necesarios para la correcta implementación del sistema. Se detallan a continuación:

```
public String getNombre()
```

```
public String getDescripcion()
```

```
public double getPrecio()
```

```
public int getDuracionMinutos()
```

Devuelven el nombre, la descripción, el precio y la duración de la actividad, respectivamente.

```
public int getMaxRecursos()
```

```
public int getMaxComentarios()
```

Devuelven el número máximo de recursos y de comentarios que puede contener la actividad.

```
public String[] getRecursos()
```

```
public String[] getComentarios()
```

Devuelven las listas de recursos y de comentarios de la actividad.

```
public boolean recursosCompleto()
```

```
public boolean comentariosCompleto()
```

Devuelven true si la actividad tiene el número máximo de recursos o de comentarios, respectivamente, false en otro caso.

```
public int getNumRecursos()
```

```
public int getNumComentarios()
```

Devuelven el número de recursos y de comentarios de la actividad, respectivamente.

2.2.2 El catálogo de actividades

El componente `CatalogoActividades` es el encargado de almacenar un conjunto de actividades. Debe proporcionar métodos para agregar, buscar y eliminar actividades, así como para guardar y cargar actividades desde archivos. A continuación, nos centramos en cada una de las funcionalidades que debe implementar este componente.

Constructor El constructor de la clase recibe como parámetro el número máximo de actividades que puede contener el catálogo.

```
public CatalogoActividades(int maxActividades)
```

Métodos auxiliares El componente `CatalogoActividades` debe proporcionar los métodos auxiliares que son necesarios para la correcta implementación de los otros componentes, los cuales se detallan a continuación:

```
public int getNumActividades()
```

Devuelve el número de actividades que contiene el catálogo.

```
public boolean actividadesCompletas()
```

Devuelve `true` si el catálogo de actividades está completo, es decir, si contiene el número máximo de actividades, `false` en otro caso.

Agregar actividad La funcionalidad que nos permite agregar una actividad se encapsula en el método:

```
public int agregarActividad(Actividad actividad)
```

que recibe como parámetro⁸ la actividad a añadir. El método devuelve un código entero que indica el resultado de la operación. La clase `CatalogoActividades` define constantes para representar los posibles resultados⁹:

Listado 5: Constantes de códigos de error en la clase `CatalogoActividades`.

⁸Será tu responsabilidad comprobar si la actividad es un valor nulo.

⁹Al igual que en `Actividad`, este enfoque separa la lógica de negocio de la presentación.

```
1 public static final int EXITO = 0;  
2 public static final int ERROR_ACTIVIDAD_NULL = 1;  
3 public static final int ERROR_DEMASIADOS = 2;
```

Si se intenta añadir una actividad a un catálogo que ya está completo, el método devuelve `ERROR_DEMASIADOS`. Es responsabilidad de la clase `InterfazUsuario` interpretar este código y mostrar el mensaje apropiado al usuario:

Listado 6: Mensaje de error mostrado por `InterfazUsuario` al interpretar `ADD_DEMASIADOS`.

```
1 No se pueden añadir más actividades.
```

El mecanismo de inserción de actividades en el catálogo no está especificado, por lo que puedes elegir la estructura de datos que consideres más adecuada para almacenar las actividades. Ten en cuenta que el catálogo de actividades tiene un tamaño fijo, que se especifica como parámetro de ejecución. Además, ten en cuenta que en el catálogo se permite borrar actividades.

Eliminar actividad Nos permite eliminar una actividad y se encapsula en el método:

```
public boolean eliminarActividad(Actividad seleccionada)
```

que recibe como parámetro la actividad a eliminar. El método devuelve `true` si la actividad se encontraba en el catálogo y fue eliminada correctamente, y `false` en caso contrario¹⁰.

¹⁰ Encontrar una actividad significa que son **el mismo** objeto.

Buscar actividad Nos permite buscar una actividad y se encapsula en el método:

```
public Actividad[] buscarActividadPorNombre(String texto)
```

Recibe como parámetro el texto a buscar en el nombre de las actividades. El método devuelve un array con aquellas actividades que contienen el texto de búsqueda como subcadena¹¹ del nombre. Esta búsqueda no diferencia mayúsculas de minúsculas. Si no se encuentra ninguna actividad que cumpla con el criterio de búsqueda, el método debe devolver un array vacío.

¹¹ Puedes usar el método `contains` de la clase `String`

Guardar y cargar actividades Para guardar y cargar las actividades a partir de un fichero de texto se usa una representación textual **simplificada** de las actividades. El proceso de guardado de las actividades se realiza en el método:

```
public void guardarActividades(String nombreArchivo)
```

Recibe como parámetro el nombre del archivo¹² en el que se guardarán las actividades. Hay que usar el método `toRawString` de la clase `Actividad` para obtener la representación textual de cada actividad, y escribir esta información en el archivo de texto.

¹²En el caso de que el fichero a escribir ya exista, se **sobreescribirá** su contenido

El proceso de carga de las actividades se realiza en el método

```
public void cargarActividades(String nombreArchivo, int maxRecursos, int maxComentarios)
```

Este método recibe como parámetro el nombre del archivo del que se cargarán las actividades. El método debe leer el contenido del archivo de texto, crear las actividades correspondientes a partir de la información leída y añadirlas al catálogo.

Es importante tener en cuenta que el método de carga debe ser capaz de cargar actividades de un archivo que haya sido guardado previamente con el método para escribirlas en el fichero. Además, el método debe ser capaz de cargar actividades de un archivo que contenga un número de actividades superior al tamaño del catálogo, en cuyo caso se deben cargar únicamente las primeras actividades que quepan en el catálogo. De la misma forma, cuando una actividad tenga más comentarios o más recursos que los especificados por parámetro la información se truncará.

Al tratarse de operaciones con ficheros, es necesario manejar las excepciones que puedan surgir durante la lectura y escritura de los archivos. Sin embargo, no es necesario que se capturen las excepciones en el propio método. En su lugar, se deben lanzar las excepciones para que sean manejadas en el método que llama a `guardarActividades` y `cargarActividades`.

2.2.3 El viaje

El componente `Viaje` es el encargado de planificar las *actividades* de un itinerario de duración variable (en días), garantizando que **no se solapen** actividades **dentro del mismo día**

y que estas puedan realizarse dentro de **los días del viaje**. También permite visualizar un resumen del viaje en el que se muestran los días, las actividades y el precio total.

Además, permite **guardar** el plan en un archivo a modo de resumen.

Constructor El constructor de la clase recibe como parámetro el número de días que durará el viaje y el máximo de actividades que se podrán hacer por día. El constructor debe inicializar la estructura o las estructuras de datos necesarias para almacenar las actividades de cada día del viaje. Por ejemplo, si el viaje dura 7 días y hay un máximo de 4 actividades por días, podemos llegar a tener 28 actividades.

```
public Viaje(int numDias, int maxActividades)
```

Planificar actividades La funcionalidad para planificar una actividad en un día concreto del viaje se encapsula en el método:

```
public int agregarActividad(int dia, Actividad actividad, String horaInicio)
```

que recibe como parámetros:

- dia: índice del día del viaje (0 es el primer día).
- actividad: la actividad a programar (con nombre, descripción, precio, duración, etc.).
- horaInicio: hora de comienzo de la actividad en formato "HH:MM" (por ejemplo, "09:30").

El método devuelve un código entero que indica el resultado de la operación. La clase Viaje define constantes para representar los posibles resultados¹³:

Listado 7: Constantes de códigos de error en la clase Viaje.

```
1 public static final int EXITO = 0;  
2 public static final int ERROR_DIA_INVALIDO = 1;  
3 public static final int ERROR_DIA_COMPLETO = 2;  
4 public static final int ERROR_SOLAPAMIENTO = 3;
```

La **duración** de la actividad (almacenada en minutos en el objeto Actividad) junto con horaInicio determinan su intervalo temporal [inicio, fin)¹⁴, donde fin = horaInicio + duracion. Para el día indicado:

¹³ Al igual que en Actividad y CatalogoActividades, este enfoque separa la lógica de negocio de la presentación.

¹⁴ Por ejemplo si comenzamos a las 10:00 y dura 60 minutos. Fin será 10:00+60 = 11:00 pero estando a las 11:00 libres.

1. Las actividades ya planificadas se ordenan por `horaInicio`.
2. Se comprueba que el nuevo intervalo `[ inicioNuevo, finNuevo)` **no se solape** con ninguno de los intervalos existentes. Es decir, ningún minuto puede pertenecer a más de una actividad. Dos actividades son compatibles si y solo si $finA \leq inicioB$ o $finB \leq inicioA$.
3. Si se detecta algún error (día inválido, día completo o solapamiento), la inserción se **rechaza** y se devuelve el código de error correspondiente; en caso contrario, la actividad se inserta en la posición correspondiente manteniendo el orden, devolviendo `EXITO`.

Para simplificar la implementación, se representan las horas como cadenas de texto en formato "HH:MM" y se convertirán a minutos desde las 00:00 para facilitar las comparaciones. Por ejemplo, "09:30" serían 570 minutos. El componente Utilidades proporciona métodos para realizar estas conversiones (`horaAMinutos` y `minutosAHora`).

Es responsabilidad de la clase `InterfazUsuario` interpretar estos códigos y mostrar los mensajes apropiados al usuario:

Listado 8: Mensajes de error mostrados por `InterfazUsuario` al interpretar los códigos de error.

-
- | | |
|---|---|
| 1 | Día inválido. |
| 2 | No se pueden agregar más actividades a este día. |
| 3 | La actividad se solapa con otra actividad ya planificada. |
-

El viaje tiene una duración total `numDias` fijada al construir `Viaje`; por tanto, los días válidos son 0 a `numDias - 1`. Si se intenta agregar una actividad en un día inválido, el método devuelve `ERROR_DIA_INVALIDO`. Si el día ya contiene el máximo de actividades permitidas, devuelve `ERROR_DIA_COMPLETO`. Si la actividad se solapa con otra ya planificada, devuelve `ERROR_SOLAPAMIENTO`.

Representación textual del itinerario Para mostrar el itinerario por pantalla, se debe proporcionar una representación textual que indique, **por cada día**, las actividades planificadas **ordenadas por hora de inicio**. También incluye un resumen de la información del viaje. El método que genera esta representación es:

```
public String toString()
```

Este método devuelve una cadena con el itinerario en el siguiente formato¹⁵ (ejemplo con 7 días):

¹⁵Ten muy en cuenta los espacios y saltos de línea en la salida.

Listado 9: Formato de representación textual del itinerario (ejemplo)

```
1 -----
2 Día 1
3 -----
4 09:30 Visita Museo
5 12:30 Cata de vinos
6
7 -----
8 Día 2
9 -----
10 12:00 Kayak
11 16:00 Playa
12
13 -----
14 Día 3
15 -----
16 (No hay actividades)
17
18 -----
19 Día 4
20 -----
21 08:00 Excursión montaña
22
23 -----
24 Día 5
25 -----
26 (No hay actividades)
27
28 -----
29 Día 6
30 -----
31 (No hay actividades)
32
33 -----
34 Día 7
35 -----
36 20:00 Cena típica
37
38 -----
39 Resumen:
40 - Días: 7
41 - Actividades: 6
42 - Precio: 250.50 €
```

Convenciones de formateo:

- Cada sección corresponde a un día del viaje (encabezado y separador explícitos).
- Las actividades de cada día se listan verticalmente bajo su encabezado de día, en orden cronológico.
- Cada actividad se muestra como HH:MM Nombre.
- Si un día no tiene actividades, se indica explícitamente con (No hay actividades).
- Al final se muestra el resumen con el número de días, de actividades y el precio total.

Guardar itinerario Para guardar el itinerario en un archivo de texto se emplea una representación **simplificada y compacta**. La generación de dicha representación se realiza directamente en el método de guardado:

```
public void guardarItinerario(String nombreArchivo)
```

Recibe el nombre del archivo donde se almacenará el plan. El método **sobrescribe** el archivo si ya existe¹⁶. El formato propuesto es:

¹⁶En caso de ya existir, se **sobrescribe** su contenido.

Listado 10: Formato de representación textual simplificada y compacta del itinerario (ejemplo)

```
1 Día 1: 09:30 Visita Museo (dur 1h 30min, 15.00 €); 12:30 Cata de vinos (dur 1h, 12.50 €)
2 Día 2: 12:00 Kayak (dur 2h, 35.00 €); 16:00 Playa (dur 3h, 0.00 €)
3 Día 3: ---
4 Día 4: ---
5 Día 5: 10:00 Ruta en bici (dur 4h, 25.00 €); 19:30 Concierto (dur 2h, 40.00 €)
6 Día 6: ---
7 Día 7: ---
8 Resumen: Días: 7; Actividades: 6; Precio total: 127.50 €
```

Reglas del formato simplificado:

- Una línea por día con el prefijo Día <num>:.
- Si hay actividades, se listan *ordenadas* por horaInicio y *separadas* por ; (punto y coma).
- Cada actividad incluye (dur XhYm, PVP€) además del nombre para facilitar lectura rápida.¹⁷
- Si un día no tiene actividades, se escribe ---.

¹⁷Hay una función para representar el formato de duración en Utilidades

Cargar itinerario En esta práctica no se contempla esta funcionalidad.

Métodos auxiliares Además de los métodos anteriores, el componente Viaje debe proporcionar los métodos auxiliares que son necesarios para la correcta implementación de los otros componentes y se detallan a continuación:

```
public int getNumDias()
```

Devuelve el número de días del viaje.

```
public Actividad[] obtenerActividadesDia(int dia)
```

Devuelve un array con las actividades planificadas para el día indicado, **ordenadas por hora de inicio**. Si el día no tiene actividades, devuelve un array vacío.

El mecanismo de almacenamiento de las actividades planificadas no está especificado, por lo que puedes elegir la estructura de datos que consideres más adecuada. Una opción sencilla es utilizar un array de arrays: un array principal con tantas posiciones como días tiene el viaje, donde cada posición contiene otro array con las actividades de ese día. Alternativamente, puedes usar un array bidimensional con un tamaño máximo de actividades por día. Si usas arrays de arrays, puedes inicializar el array principal en el constructor y crear los arrays internos

```
public boolean eliminarActividad(int dia, String horaInicio)
```

Elimina la actividad planificada en el día y hora indicados. Devuelve true si se ha eliminado correctamente y false en otro caso (por ejemplo, si no existe ninguna actividad en ese día y hora).

```
public int getNumActividadesDia(int dia)
```

Devuelve el número de actividades planificadas para el día indicado.

Estas operaciones facilitan la implementación de `InterfazUsuario` (menú en consola). El componente `Utilidades` proporciona métodos auxiliares para validación de entrada de datos, conversión de horas y gestión de días que serán útiles en la implementación de este componente.

2.2.4 La interfaz de usuario

La interfaz de usuario es el componente que se encarga de gestionar la interacción con el usuario. Esta es la **única clase del sistema** responsable de mostrar información por pantalla y solicitar datos al usuario¹⁸.

Debe mostrar un menú con las opciones disponibles y permitir al usuario seleccionar una de ellas. Las opciones disponibles son las siguientes:

1. Agregar Actividad
2. Consultar/Editar Actividad
3. Guardar Actividades
4. Cargar Actividades
5. Planificar Viaje
6. Guardar Itinerario
7. Salir

El componente se encargará de llamar a los métodos correspondientes de los otros componentes para realizar las acciones solicitadas por el usuario, además de encargarse de la entrada y salida de los datos necesarios para realizar dichas acciones.

Por ejemplo, si el usuario selecciona la opción de agregar actividad, la interfaz de usuario se encargará de solicitar la información de la nueva actividad al usuario y llamará al método

¹⁸ Las clases `Actividad`, `CatalogoActividades` y `Viaje` devuelven códigos de error que `InterfazUsuario` interpreta para mostrar los mensajes apropiados.

correspondiente del componente que gestiona el catálogo de actividades. La interfaz interpreta los códigos de retorno y muestra los mensajes apropiados.

Una vez realizada la acción solicitada por el usuario, la interfaz de usuario debe volver a mostrar el menú de opciones. El programa debe seguir ejecutándose hasta que el usuario seleccione la opción de salir. Este bucle de ejecución se conoce como el bucle principal del programa, y se define en el método:

```
private void menuPrincipal(Scanner scanner)
```

A continuación se puede ver un ejemplo de cómo se debe mostrar el menú de opciones al usuario (observa que el menú se muestra en la consola, que el usuario debe introducir un número para seleccionar una opción y que se muestra un símbolo¹⁹ para representar los espacios en blanco _):

Listado 11: Menú principal y petición de acción al usuario

```
1 ---_Menú_Principal_---
2 1._Agregar_Actividad
3 2._Consultar/Editar_Actividad
4 3._Guardar_Actividades
5 4._Cargar_Actividades
6 5._Planificar_Viaje
7 6._Guardar_Itinerario
8 7._Salir
9 Elige_una_opción:
```

¹⁹Se muestran los espacios en blanco de manera explícita usando el símbolo concreto _ para que puedas generar una salida exactamente igual.

A continuación, se detalla la funcionalidad que debe implementar la interfaz de usuario para cada una de las opciones del menú.

Agregar Actividad La funcionalidad que permite agregar una actividad se implementa en el método:

```
private void agregarActividad(Scanner scanner)
```

Recibe como parámetro un objeto de la clase Scanner para leer la entrada del usuario. El método debe solicitar al usuario el nombre de la actividad. A continuación, debe solicitar al usuario que introduzca la descripción, el precio, la duración en minutos, los recursos y los comentarios de la actividad. Una vez introducidos los datos, el método debe crear un objeto de la clase Actividad con la información proporcionada y añadir la actividad al catálogo de actividades.

El método interpreta los códigos de error devueltos por `Actividad.agregarRecurso()`, `Actividad.agregarComentario()` y `CatalogoActividades.agregarActividad()` para mostrar los mensajes apropiados al usuario. Si la actividad se añade correctamente, se debe mostrar un mensaje de éxito²⁰:

²⁰ Recuerda que los casos de prueba esperan estos mensajes en concreto, por lo que tienes que ser preciso con espacios y signos de puntuación.

Listado 12: Ejemplo de funcionalidad Agregar actividad

```
1 Nombre de la actividad: Museo del Prado
2 Descripción: Visita guiada al museo más importante de Madrid
3 Precio (€): 15.50
4 Duración (minutos): 120
5 Introduce los recursos (una línea por recurso, escribe 'fin' para terminar):
6 Entrada con descuento
7 Audio guía incluida
8 fin
9 Introduce los comentarios (una línea por comentario, escribe 'fin' para terminar):
10 Muy interesante
11 Gran colección de arte
12 fin
13 ¡Actividad agregada exitosamente!
```

Fíjate en que el usuario debe introducir los recursos y comentarios de la actividad, una línea por cada uno, y escribir `fin` para indicar que ha terminado de introducir los datos. En el caso de que la actividad no se haya podido añadir al catálogo, se debe mostrar el error `No se pueden añadir más actividades.` por pantalla²¹. En cualquier caso, el método debe volver al menú principal una vez finalizada la acción.

²¹ Esto se escribirá cuando el método de agregar Actividad al catálogo devuelva `ADD_ACTIVIDAD_NULL`. Ocurrirá cuando se pase una Actividad nula.

Consultar/Editar Actividad La funcionalidad que permite consultar y editar una actividad se implementa en el método:

```
private void consultarActividad(Scanner scanner)
```

Recibe como parámetro un objeto de la clase `Scanner` para leer la entrada del usuario. El método debe solicitar al usuario el texto a buscar dentro del nombre de las actividades que desea consultar o editar²². Una vez realizada la consulta, se le muestra al usuario el listado de actividades resultante y se le pide que seleccione una de ellas. A continuación, se le muestra la información de la actividad seleccionada y se le muestra el menú de edición de actividad, que permite añadir un recurso, un comentario o eliminar la actividad:

²² Si el usuario introduce `-FIN-` se cancela la búsqueda.

Listado 13: Ejemplo de funcionalidad Consultar/Editar actividad

```
1 Introduce el texto de la actividad a buscar (-FIN- para volver): museo
2 Actividades encontradas:
3 1. Museo del Prado
4 2. Museo Reina Sofía
5 Elige una actividad: 1
6
7 Actividad: Museo del Prado
8 Descripción: Visita guiada al museo más importante de Madrid
9 Precio: 15.50 €
10 Duración: 2h
11 Recursos:
12 - Entrada con descuento
13 - Audio guía incluida
14 Comentarios:
15 1. Muy interesante
16 2. Gran colección de arte
17
18 1. Añadir recurso
19 2. Añadir comentario
20 3. Eliminar actividad
21 4. Volver
22 Elige una opción: 1
23 Introduce el recurso a añadir: Mapa del museo
24 Recurso añadido exitosamente.
```

Como se puede observar, el usuario puede seleccionar una actividad de la lista de actividades encontradas y, a continuación, puede añadir un recurso, un comentario o eliminar la actividad. Una vez realizada la acción solicitada por el usuario, el método debe volver al menú principal. Las acciones de añadir recurso y añadir comentario interpretan los códigos de error devueltos por los métodos de Actividad y muestran los mensajes correspondientes.

Métodos auxiliares de Consultar/Editar La funcionalidad de este método está, a su vez, encapsulada en otros métodos auxiliares que se detallan a continuación:²³

private Actividad **buscarActividadPorNombre**(Scanner scanner)

Método que recibe como parámetro un objeto de la clase Scanner para leer la entrada del usuario y devuelve la actividad seleccionada por el usuario. El método debe solicitar al usuario el texto a buscar dentro del nombre de las actividades

²³Estos métodos se apoyan en los métodos ya desarrollados pero incorporan la interacción con el usuario.

que desea consultar o editar y mostrar el listado de actividades resultante²⁴.

²⁴Si la búsqueda no arroja resultados, hay que solicitar al usuario un nuevo término de búsqueda.

A continuación, debe solicitar al usuario que seleccione una de las actividades encontradas y devolver la actividad seleccionada. La selección de las actividades también está encapsulada en otro método:

```
private Actividad seleccionarActividad(Scanner scanner, Actividad[] actividades)
```

Recibe como parámetro un objeto de la clase Scanner para leer la entrada del usuario y un array de actividades. El método debe mostrar al usuario el listado de actividades y solicitar al usuario que seleccione una de ellas, mediante el número de orden en el listado (ver Listado 13). A continuación, debe devolver la actividad seleccionada por el usuario.

Por último, el método

```
private void editarActividad(Scanner scanner, Actividad seleccionada)
```

que recibe como parámetros un objeto de la clase Scanner para leer la entrada del usuario y la actividad seleccionada por el usuario. El método debe mostrar la información de la actividad seleccionada y el menú de edición de actividad, que permite añadir un recurso, un comentario o eliminar la actividad (ver Listado 13). En el caso de eliminar una actividad, se debe mostrar el mensaje Actividad eliminada. por pantalla:

Listado 14: Ejemplo de eliminar una actividad

```
1 Introduce el texto de la actividad a buscar (-FIN- para volver): prado
2 Actividades encontradas:
3 1. Museo del Prado
4 Elige una actividad: 1
5
6 Actividad: Museo del Prado
7 Descripción: Visita guiada al museo más importante de Madrid
8 Precio: 15.50 €
9 Duración: 2h
10 Recursos:
11 - Entrada con descuento
12 - Audio guía incluida
13 - Mapa del museo
14 Comentarios:
15 1. Muy interesante
16 2. Gran colección de arte
```

```
17
18 1. Añadir recurso
19 2. Añadir comentario
20 3. Eliminar actividad
21 4. Volver
22 Elige una opción: 3
23 Actividad eliminada.
```

Planificar Viaje La funcionalidad que permite planificar las actividades para cada día del viaje se implementa en el método:

```
private void planificarViaje(Scanner scanner)
```

Recibe como parámetro un objeto de la clase Scanner para leer la entrada del usuario. El método debe, en primer lugar, imprimir por pantalla la planificación actual del viaje. Después, debe solicitar al usuario el día del viaje en el que desea planificar la actividad, la hora de inicio y la actividad que desea añadir. Una vez introducidos los datos, el método debe llamar a `Viaje.agregarActividad()` e interpretar el código de retorno para mostrar el mensaje apropiado:

Listado 15: Ejemplo de funcionalidad Planificar viaje

```
1 Planificación del viaje:
2 -----
3 Día 1
4 -----
5 (No hay actividades)
6
7 -----
8 Día 2
9 -----
10 (No hay actividades)
11
12 -----
13 Día 3
14 -----
15 (No hay actividades)
16
17 Introduce el día del viaje (1-3): 1
18 Introduce la hora de inicio (HH:MM): 09:30
19 Introduce el texto de la actividad a buscar (-FIN- para volver): museo
20 Actividades encontradas:
21 1. Museo del Prado
```


- 22 Elige una actividad: 1
 - 23 Actividad planificada para el día 1 a las 09:30
-

Para pedir al usuario el día del viaje y la hora de inicio, se debe utilizar el componente Utilidades (ver section 2.2.5). En cualquier caso, el método debe volver al menú principal una vez finalizada la acción.

Si la actividad no se puede planificar, la interfaz interpreta los códigos de error devueltos por Viaje y muestra el mensaje correspondiente:

- ERROR_DIA_INVALIDO: “Día inválido.”
- ERROR_DIA_COMPLETO: “No se pueden agregar más actividades a este día.”
- ERROR_SOLAPAMIENTO: “La actividad se solapa con otra actividad ya planificada.”

Guardar Actividades La funcionalidad que permite guardar las actividades en un archivo de texto se implementa en el método

```
private void guardarActividades(Scanner scanner)
```

Recibe como parámetro un objeto de la clase Scanner para leer la entrada del usuario. El método debe solicitar al usuario la ruta del archivo en el que desea guardar las actividades y llamar al método auxiliar privado guardarActividadesEnArchivo() que encapsula la operación de guardado. Si las actividades se guardan correctamente, se debe mostrar un mensaje²⁵ de éxito:

²⁵En caso de error, el mensaje a mostrar será:
Error al guardar el archivo.

Listado 16: Ejemplo de funcionalidad Guardar actividades

- 1 Archivo donde guardar las actividades: actividades.txt
 - 2 Actividades guardadas en actividades.txt
-

En cualquier caso, el método debe volver al menú principal una vez finalizada la acción.

Cargar Actividades La funcionalidad que permite cargar las actividades desde un archivo de texto se implementa en el método:

```
private void cargarActividades(Scanner scanner)
```

Recibe como parámetro un objeto de la clase Scanner para leer la entrada del usuario. El método debe solicitar al usuario la ruta del archivo del que desea cargar las actividades y llamar al método auxiliar privado `cargarActividadesDesdeArchivo()` que encapsula la operación de carga. Si las actividades se cargan correctamente, se debe mostrar un mensaje de éxito:

En caso de error, el mensaje a mostrar será: *Error al cargar el archivo.*

Listado 17: Ejemplo de funcionalidad Cargar actividades

- 1 Archivo de donde cargar las actividades: `actividades.txt`
- 2 Actividades cargadas desde `actividades.txt`

En cualquier caso, el método debe volver al menú principal una vez finalizada la acción.

Guardar Itinerario La funcionalidad que permite guardar el itinerario del viaje en un archivo de texto se implementa en el método:

```
private void guardarItinerario(Scanner scanner)
```

Recibe como parámetro un objeto de la clase Scanner para leer la entrada del usuario. El método debe solicitar al usuario la ruta del archivo en el que desea guardar el itinerario y llamar al método auxiliar privado `guardarItinerarioEnArchivo()` que encapsula la operación de guardado. Si el itinerario se guarda correctamente, se debe mostrar un mensaje de éxito:

En caso de error, el mensaje a mostrar será: *Error al guardar el archivo.*

Listado 18: Ejemplo de funcionalidad Guardar itinerario

- 1 Archivo donde guardar el itinerario: `itinerario.txt`
- 2 Itinerario guardado en `itinerario.txt`

En cualquier caso, el método debe volver al menú principal una vez finalizada la acción.

2.2.5 Utilidades

El componente Utilidades proporciona métodos de utilidad para la entrada de datos por teclado y para la conversión de formatos.

Nota importante: Los métodos de entrada de datos (`leerCadena`, `leerNumero`, `leerDouble`, `leerHora`) muestran mensajes de validación por pantalla cuando el usuario introduce datos incorrectos. Esto es una excepción justificada al principio de que

solo InterfazUsuario escribe por pantalla, ya que estos métodos son utilidades de bajo nivel para la interacción directa con el usuario durante la entrada de datos²⁶.

En concreto, se deben implementar los siguientes métodos:

```
public static String leerCadena(Scanner teclado, String s)
```

Método que muestra el mensaje `s` por pantalla y lee una cadena de caracteres introducida por el usuario. El método devuelve la cadena de caracteres leída.

```
public static int leerNumero(Scanner teclado, String mensaje, int minimo, int maximo)
```

Método que muestra el mensaje `mensaje` por pantalla y lee un número entero introducido por el usuario. El método debe comprobar que el número introducido está en el rango `[minimo, maximo]` y, si no es así, volver a solicitar el número²⁷. El método devuelve el número entero leído.

```
public static double leerDouble(Scanner teclado, String mensaje, double minimo, double maximo)
```

Método que muestra el mensaje `mensaje` por pantalla y lee un número decimal introducido por el usuario. El método debe comprobar que el número introducido está en el rango `[minimo, maximo]` y, si no es así, volver a solicitar el número²⁸. El método devuelve el número decimal leído.

```
public static String leerHora(Scanner teclado, String mensaje)
```

Método que muestra el mensaje `mensaje` por pantalla y lee una hora introducida por el usuario en formato "HH:MM". El método debe validar que el formato sea correcto (dos dígitos para la hora, dos puntos (:), y dos dígitos para los minutos) y que la hora esté en el rango válido (00:00 a 23:59). Si el formato no es correcto o la hora no es válida, debe volver a solicitar la hora²⁹. El método devuelve la cadena de caracteres con la hora en formato "HH:MM". Los métodos de conversión de horas (`horaAMinutos` y `minutosAHora`) son especialmente útiles, ya que facilitan la comparación de intervalos temporales trabajando con enteros en lugar de cadenas de texto.

```
public static int horaAMinutos(String hora)
```

²⁶Esta decisión de diseño simplifica el código y mejora la experiencia del usuario al proporcionar retroalimentación inmediata sobre errores de entrada.

²⁷Si el número no está en el rango válido, el método muestra el mensaje: "El número debe estar entre [minimo] y [maximo]." Si la entrada no es un número válido, muestra: "Por favor, introduce un número válido."

²⁸Los mensajes de validación son análogos a los del método `leerNumero`.

²⁹El método muestra mensajes específicos de validación: "Formato incorrecto. Usa el formato HH:MM (por ejemplo, 09:30).", "Las horas deben estar entre 00 y 23.", o "Los minutos deben estar entre 00 y 59."

Método que recibe como parámetro una hora en formato "HH:MM" (por ejemplo, "09:30") y devuelve el número de minutos transcurridos desde las 00:00. Por ejemplo, "09:30" devolvería 570 ($9 \times 60 + 30$).

```
public static String minutosAHora(int minutos)
```

Método que recibe como parámetro un número de minutos transcurridos desde las 00:00 y devuelve la hora correspondiente en formato "HH:MM". Por ejemplo, 570 minutos devolvería "09:30".

```
public static String formatearDuracion(int duracionMinutos)
```

Método que recibe una duración expresada en minutos y devuelve una cadena con formato legible. Por ejemplo, si la duración es 60, el método devolverá "1h"; si es 90, devolverá "1h 30min"; y si es 30, devolverá "30min".

```
public static String formatearPrecio(double precio)
```

Método que recibe un valor decimal con el precio y devuelve una cadena formateada con dos decimales y el símbolo del euro. Por ejemplo, el valor 12.5 se devolverá como "12.50 €".

```
public static double cadenaAPrecio(String precioStr)
```

Método que recibe una cadena con un precio formateado (por ejemplo, "12.50 €") y devuelve su valor numérico como double. El método elimina el símbolo del euro y los espacios antes de realizar la conversión.

2.3 La función principal main

La función principal main es la encargada de inicializar los componentes de la aplicación y de gestionar la ejecución del programa.

También se encarga de recibir los argumentos de la línea de comandos que configuran el funcionamiento de la aplicación. En concreto, la función principal debe recibir los siguientes argumentos:

Tabla 3: Argumentos de la línea de comandos (y su orden)

#	Parámetro	Descripción
1	maxRecursosPorActividad	Número máximo de recursos que puede contener una actividad.
2	maxComentariosPorActividad	Número máximo de comentarios que puede contener una actividad.
3	maxActividadesEnCatalogo	Número máximo de actividades que puede contener el catálogo de actividades.
4	numDiasViaje	Número de días del viaje.
5	maxActividadesPorDia	Número máximo de actividades por día del viaje.
6	nombreArchivoActividades	(Opcional) Nombre del archivo de texto con las actividades a cargar al inicio.

Importancia de los argumentos en la línea de comandos

Los argumentos proporcionados mediante la línea de comandos permiten a los programas recibir configuraciones o datos de entrada directamente al momento de su ejecución, sin necesidad de modificar el código fuente o interactuar manualmente con el usuario tras iniciarlo. Esto tiene varias ventajas clave:

1. **Flexibilidad y reutilización:** usar argumentos en la línea de comandos hace que los programas sean más versátiles, ya que pueden adaptarse fácilmente a diferentes escenarios sin necesidad de cambios en el código. Por ejemplo, un programa que gestione viajes puede aceptar diferentes configuraciones de días y actividades como argumentos para adaptarse a viajes de distintas duraciones.
2. **Automatización:** los argumentos facilitan la integración de programas en scripts o procesos automatizados. Al permitir que los datos se pasen directamente al programa, se elimina la necesidad de interacción manual, lo cual es ideal para entornos de producción o tareas repetitivas.
3. **Cumplimiento de buenas prácticas:** siguiendo estándares comunes, como los de herramientas Unix, el uso de argumentos fomenta una interfaz coherente y predecible para los usuarios, lo cual es especialmente importante en software diseñado para otros desarrolladores.
4. **Simulación de escenarios reales:** enseñar a usar ar-

gumentos en programas educativos permite a los estudiantes familiarizarse con prácticas comunes en la industria, donde los sistemas suelen depender de configuraciones y parámetros proporcionados al momento de la ejecución.

5. **Claridad y separación de responsabilidades:** pasar argumentos en la línea de comandos promueve la separación entre la lógica de la aplicación y su configuración. Esto facilita el desarrollo, el mantenimiento y las pruebas, ya que los valores específicos pueden cambiarse sin necesidad de alterar el código fuente.

Fíjate que el argumento `nombreArchivoActividades` es opcional, lo que significa que el programa puede ejecutarse sin proporcionar este argumento. En caso de que se proporcione, el programa debe cargar las actividades desde el archivo especificado al inicio. Si no se proporciona, el programa debe comenzar sin actividades cargadas.

La función principal debe inicializar los componentes de la aplicación y ejecutar el bucle principal del programa, que se define en el método `menuPrincipal` de la interfaz de usuario (ver section 2.2.4). La función principal debe gestionar las excepciones que puedan surgir durante la ejecución del programa y mostrar un mensaje de error en caso de que ocurra una excepción no controlada.

2.4 Casos de prueba

Para que puedas comprobar que tu implementación es correcta, se proporcionan una serie de casos de prueba que puedes utilizar para verificar el funcionamiento de tu programa. Además, introduce la metodología de desarrollo dirigido por pruebas³⁰ (*Test-Driven Development, TDD*) para implementar cada funcionalidad de forma incremental y asegurarte de que cada parte de tu programa funciona correctamente antes de pasar a la siguiente. Aunque implementar y diseñar los casos de prueba queda fuera del alcance de esta asignatura, es importante que conozcas su utilidad y cómo se utilizan. En este caso, los casos de prueba se especifican usando código Java en ficheros que contienen una serie de comandos y mensajes esperados que se pueden utilizar para verificar el correcto funcionamiento de tu programa.

³⁰Dave Astels. *Test Driven development: A Practical Guide*. Prentice Hall Professional Technical Reference. DOI: [10.5555/864016](https://doi.org/10.5555/864016)

3 Criterios de evaluación

Para llevar a cabo la evaluación de la práctica se tendrán en cuenta varios aspectos complementarios que se detallan a continuación.

3.1 Casos de prueba

Se proporcionan una serie de casos de prueba que se utilizarán para comprobar el correcto funcionamiento de tu programa. Estos casos de prueba se han diseñado para cubrir diferentes situaciones y comprobar que tu implementación es correcta. Es importante que tu programa pase todos los casos de prueba para asegurar que cumple con los requisitos y funcionalidades solicitadas.

! Importante Durante la evaluación, se utilizarán casos de prueba adicionales que no se proporcionan en el enunciado. Estos casos de prueba adicionales se utilizarán para comprobar que tu implementación es correcta y cumple con los requisitos solicitados. El hecho de que tu programa pase los casos de prueba es condición necesaria, pero no suficiente, para obtener una calificación alta. Además de pasar los casos de prueba, tu implementación se evaluará con el resto de criterios detallados en esta sección.

3.2 Diseño y estructura del código

Se valorará la claridad, organización y estructura del código fuente de tu programa. Es importante que tu implementación sea fácil de leer, comprender y mantener. Se valorará positivamente la correcta nomenclatura de variables y métodos, la división en métodos y clases coherentes y la reutilización de código. Se valorará el uso de buenas prácticas de programación y el cumplimiento de los principios de diseño y desarrollo de software. Es importante que tu implementación siga las recomendaciones y buenas prácticas de programación en Java, así como que siga los principios de diseño y desarrollo de software.

! Importante Se valorará negativamente:

- Código duplicado o redundante.
- Métodos o clases demasiado largos o complejos.
- Falta de modularidad y reutilización de código.
- Feb 10, 2026
- Uso incorrecto de estructuras de control: bucles y condicionales.
- Mal uso de excepciones.

3.3 Documentación y comentarios

Se valorará la calidad de la documentación y los comentarios en el código fuente de tu programa. Es importante que tu implementación esté correctamente documentada y que los comentarios sean claros y concisos. Se valorará positivamente la presencia de comentarios que expliquen el propósito de las clases y métodos, así como el significado de variables y parámetros. Al igual que en el diseño y estructura del código, se valorará negativamente la falta de comentarios y documentación, así como la presencia de comentarios innecesarios o confusos.

3.4 Errores graves

A continuación, se detallan una serie de errores graves que, de estar presentes en tu implementación, **supondrán un suspenso** en la calificación final de la práctica. Es importante que tengas en cuenta estos errores y los evites en tu implementación:

- No compilar: tu programa no compila correctamente y no se puede ejecutar.
- Argumentos incorrectos: tu programa no recibe los argumentos de la línea de comandos correctamente.
- Plagio: tu implementación contiene código copiado de otras fuentes.
- Código malicioso: tu implementación contiene código malicioso o perjudicial.

Referencias

- [1] Dave Astels. *Test Driven development: A Practical Guide*. Prentice Hall Professional Technical Reference. DOI: [10.5555/864016](https://doi.org/10.5555/864016).
- [2] *Licencias educativas gratuitas. Asistencia a la comunidad*. JetBrains. 2024. URL: <https://www.jetbrains.com/es-es/community/education/#students>.
- [3] Eric Redmond. *Maven in 5 Minutes – Maven*. 2008. URL: <https://maven.apache.org/guides/getting-started/maven-in-five-minutes.html>.